Defending Your Submission

Every significant decision, why you made it, and what happens when they push

- How to use this
- Problem 1 — Position Tracker
  - Decision 1: One struct instead of three maps
  - Decision 2: `find` instead of `operator[]`
  - Decision 3: `-ffp-contract=off`
  - Decision 4: Validation off by default
  - Decision 5: The flat-position average
  - Decision 6: Batch bindings
  - Concurrency — the answer they'll dig into
- Problem 2 — Ranking Engine
  - Decision 1: The bug has two modes
  - Decision 2: `!(p > 0.0)` not `p <= 0.0`
  - Decision 3: Exclude with a reason, don't rank last
  - Decision 4: The determinism tiebreak
  - Decision 5: Day-major layout
  - Decision 6: Don't parallelise the loop
- Problem 3 — Snapshot Cache
  - Decision 1: There are two levels of shared state
  - Decision 2: A read can trigger the rehash
  - Decision 3: Return by value
  - Decision 4: The ring buffer
  - Decision 5: The bug that survives correct locking
  - Decision 6: Seqlock over mutex (the Tier-2 version)
- Problem 4 — P&L and Terminology
  - The numbers
  - The two one-liners
  - The note's errors — I count five, not four
  - Decision: net delta is 1 − DF, not zero
  - Decision: total is convention-independent, the split isn't
- Problem 5 — Synthetic Future Arbitrage
  - Decision 1: `e^{+rT}`, not `e^{-rT}`
  - Decision 2: Check the neighbouring strikes first
  - Decision 3: Executable prices only
  - Decision 4: The sanity band
  - Decision 5: The verification
  - The (c) answers
- Cross-Cutting Themes
  - 1. Measure, don't assume
  - 2. Silent wrongness beats loud failure — as a danger
  - 3. Question the requirement before optimising it
  - 4. Verification has a hierarchy
  - 5. Know what you'd do differently

How to use this

They have read your submission. This is now the largest part of the interview, and it is the part you can prepare with certainty.

For each decision: **what you chose**, **why**, **the number that supports it**, and **the follow-up they'll ask**. Two interviewers — the head of quant research will probe Problems 2, 4 and 5; Deepak will probe 1 and 3.

Rule for all of it: you must be able to *derive* each answer, not recite it. If they change the premise slightly and you can't adapt, that's the failure mode.

Problem 1 — Position Tracker

Decision 1: One struct instead of three maps

What: replaced `qty` / `avg_price` / `notional` dicts with one `Position` struct per instrument.

Why: three hash lookups per fill to update three fields that always change together. And they can desynchronise — any path that updates two of three leaves the book internally inconsistent with nothing to detect it.

Number: three maps 72.9 ns/fill → one struct with string key 39.8 ns → pre-resolved id 11.1 ns.

Follow-up: "Why not keep notional as a field?" Because it's `qty × avg_price` — one multiply. Storing a derived value is a correctness liability, not an optimisation. The only reason to cache it would be if the multiply showed up in a profile, and it doesn't.

Follow-up: "Is a struct always better?" No — it's AoS, and it's right here because all three fields are touched together. If you only ever read quantities across all instruments, SoA would be better. In Problem 2 I went the *other* way for exactly that reason.

Decision 2: `find` instead of `operator[]`

What: read paths use `find`; only the write path inserts.

Why: `operator[]` default-constructs and **inserts** if the key is absent. So a read mutates the map — it grows without bound from typos and retired symbols, and the caller gets a fabricated `{0, 0.0}` indistinguishable from a genuinely flat position.

Follow-up: "The Python does `dict.get(sym, 0)` — isn't that the same?" No, and that's the trap. `dict.get` reads **without** inserting. The C++ mirror image of `[]` *does* insert. They look equivalent and behave oppositely.

Follow-up: "How would you catch this in a test?" Assert the instrument count is still zero after the read. Without that assertion the test passes on the broken code, because `average()` returns a `double` that merely happens to be NaN — it looks like a value.

Decision 3: `-ffp-contract=off`

What: pinned the flag on the target; `-Ofast` is a hard build failure.

Why: FMA fuses `a*b + c` into one operation with a **single rounding** instead of two. More accurate — which is exactly why it breaks bit-reproduction of Python's two-step arithmetic.

Numbers:

Build	Fills differing (of 3,000)
`-O2`	0
`-O2 -march=native`	**1,781**
`-Ofast -march=native`	1,659

Follow-up: "But plain `-O2` passes — so why bother?" This is the question, and the answer is the strongest part.

GCC's default is *already* `-ffp-contract=fast`. It passes only because **baseline x86-64 has no FMA instruction** — the compiler is permitted to fuse but has nothing to fuse with. Add `-march=native` and it appears. And **FMA is baseline on ARMv8**, so the same source contracts by default on Graviton or Apple Silicon.

So parity survived on two coincidences — target ISA and optimisation level — neither of which anyone chose. The flag makes it depend on a stated decision instead.

Follow-up: "Show me."

```
-O2                                mulsd, mulsd, addsd      (two roundings)
-O2 -march=native                  vmulsd, vfmadd132sd      (one rounding)
-O2 -march=native -ffp-contract=off  vmulsd, vmulsd, vaddsd   (two again)
```

Decision 4: Validation off by default

What: `onFill` calls `checkFill` on every fill but only **counts** failures — it still applies them.

Why: the question demands two conflicting things. Part (a) requires exact parity; the prototype accepts a negative price and blends it in, giving an average of −1275. Part (c) asks about bad-data handling.

Rejecting inside `onFill` would break the guarantee that makes a cutover safe: **any difference between old and new must be a bug in my port, not a fix I decided to sneak in.** Fixing bugs and changing languages in the same commit means you can never explain a number to the risk desk.

Counting resolves it, because the conflict was only ever about changing the *numbers*.

Follow-up: "Isn't shipping known-broken behaviour irresponsible?" It's temporary and visible. `badFillCount()` surfaces it by reason, so ops can see the feed is sending garbage. Once parity is signed off, turning the count into a reject is one line.

Decision 5: The flat-position average

What: reproduced the prototype's behaviour faithfully rather than fixing it.

Why it happens: the code tests `old_qty > 0` to determine side. But a position has *three* states — long, short, flat — and a boolean has two. Flat gets lumped with short:

- long → flat: `(True != False)` = True → branch 1 → avg = **fill price**
- short → flat: `(False != False)` = False → branch 3 → avg = **old average**

Same event, two different answers.

Why it's harmless: notional is `0 × avg = 0`, and the next fill sees `old_qty == 0` so it takes the fresh-open branch and starts clean.

Follow-up: "How would you fix it properly?" Make flat explicit — return `std::optional` and give `nullopt` when `qty == 0`. A flat position has no entry price; that's the honest representation. That's what the Corrected mode does.

Decision 6: Batch bindings

Numbers over 200,000 fills:

Path	ns/fill	Speedup
Pure Python	545	1.0×
C++ per-fill binding	**493**	**1.1×**
C++ list of tuples	316	1.7×
C++ batch arrays, GIL released	**104**	**5.3×**

The point: a per-fill binding gets 1.1× on a core that's 45× faster. The whole gain is handed back in boundary crossing.

Follow-up: "Why can't the tuples path release the GIL?" Every element is a Python object unpacked *inside* the loop. The arrays path converts to `std::vector` **before** the function body runs, so nothing inside touches Python.

Follow-up: "What happens if you release it wrongly?" It crashes. Not a slowdown — touching a `py::` object without the GIL is undefined.

Concurrency — the answer they'll dig into

What: the class isn't thread-safe, and I argue a mutex isn't the fix.

Why: a mutex gives **mutual exclusion but not order**, and this computation is order-dependent.

Number: the same fills across four threads under one mutex gave **30 distinct results from 30 identical runs** — on a single-core machine.

The detail that makes it land: the **share count agreed every time** because addition commutes. Only the average price and P&L differed. So a smoke test checking positions passes cleanly while the money numbers are wrong by lakhs.

The fix: shard by instrument — `hash(symbol) % numThreads`. Each instrument owned by exactly one thread, so order is preserved by construction and no lock is needed anywhere.

Follow-up: "One thread per instrument doesn't scale." It's not one thread per instrument. 4 threads, 500 instruments — each thread owns ~125. The property needed isn't "a thread each," it's "**every fill for a given instrument goes to the same thread**."

Follow-up: "What breaks with sharding?" Load imbalance. Fill rates are wildly uneven — BANKNIFTY weeklies can outweigh every cash equity combined. Two hot symbols hashing to one thread saturate it while others idle. Fixes: more shards than threads, or assign by measured volume, or accept it.

Follow-up: "Do you even need threads?" Probably not. At ~11 ns/fill one thread handles ~90 million fills/second. A single-threaded consumer fed by a lock-free SPSC queue preserves order by construction and is almost certainly enough. The requirement is that the *entry point* be safe to call from the feed thread.

Problem 2 — Ranking Engine

Decision 1: The bug has two modes

Mode 1 — `+inf`. A zero prior close gives infinity, which **sorts perfectly well**. It compares consistently with everything. So the engine reports a stock that didn't trade as today's top gainer, the signal ships, and nothing logs a complaint. **Worse than a crash** — a crash gets fixed Tuesday.

Mode 2 — NaN. `0.0/0.0` gives NaN, which breaks **strict weak ordering** — that's UB in `std::sort`, not a bad order:

```
NaN vs +5%:  neither greater  → "equivalent"
NaN vs −5%:  neither greater  → "equivalent"
+5% vs −5%:  +5% IS greater   → but these aren't equivalent!
```

Transitivity of equivalence fails, and introsort's logic depends on it.

The number that matters:

NaN returns	valid entries left out of order
72 of 500	**25**
250 of 500	**46**

Twenty-five stocks with perfectly good prices ranked wrongly. **Filtering NaN from the output wouldn't save them** — the damage happens inside the sort.

Follow-up: "Which is worse?" The infinity, because it's silent and plausible. NaN at least produces obviously broken output if anyone looks.

Decision 2: `!(p > 0.0)` not `p <= 0.0`

Why: every ordered comparison against NaN is **false**. So `p > 0.0` is false for zero, negatives *and* NaN — one comparison covers all three. But `p <= 0.0` is *also* false for NaN, so it **misses** it.

Same logic in the band check, written as `!(fabs(r) <= band)` rather than `(fabs(r) > band)` — the first excludes a NaN, the second admits it.

Follow-up: "Isn't that obscure?" It needs the comment, which it has. But the alternative — an explicit `isnan` check — adds a branch and a second thing to keep in sync. One comparison that provably covers every case is better.

Decision 3: Exclude with a reason, don't rank last

Why: a rank is a trading signal. Ranking a stock we have no valid price for — whether last, or with a substituted 0.0 — puts a fabricated number into that signal. "Flat" and "unknown" are different facts.

Every stock lands in exactly one list: `ranked.size() + excluded.size() == N`, asserted in tests. So an operator can always answer "why isn't RELIANCE in today's list?"

Follow-up: "Why four reason codes rather than one?" Because they mean different things operationally. `NoPrevClose` after a listing is a normal Monday. `BadPrevClose` is a broken feed. Same count, opposite response.

Decision 4: The determinism tiebreak

What: total order — return descending, then **id ascending**.

Why: `std::sort` is unstable, and on a quiet day dozens of stocks sit at exactly 0.00%. Their order then depends on introsort's pivoting, so **the same input ranks differently between the batch run and the daily run**, or between compiler versions.

For an engine feeding a trading signal that's unacceptable — you can never reconcile live output against a backtest.

This bug wasn't in the problem statement. Worth saying so.

Decision 5: Day-major layout

What: `close[day][stock]` instead of `close[stock][day]`.

Why: the work is **cross-sectional** — every stock, today and yesterday. Stock-major means 500 scattered 16-byte reads, each pulling a 64-byte cache line and using a quarter of it.

Number: 1,017 ns → 339 ns. **3×**, no algorithmic change.

Follow-up: "Would that always be right?" No. If the dominant pass were per-stock time series, stock-major would win. Layout follows the access pattern, and here every operation is cross-sectional.

Follow-up: "Did you get it right first time?" No — and this is worth volunteering. I then stored my own carried-forward state **stock-major**, reintroducing the exact pattern I'd just criticised. Cost ~6 µs/day. I only found it because component timings didn't add up to end-to-end.

Decision 6: Don't parallelise the loop

The arithmetic, per day:

Stage	Time	Share
return loop (what was asked)	306 ns	**1.7%**
validation	924 ns	5%
sort	**~17,000 ns**	**~90%**

Amdahl caps whole-day speedup at **1.017×**. And thread create+join costs 14–25 µs to save 0.3 µs — measured **46–82×**

slower.

Parallelise days instead: 2,500 of them, ~18 µs each, and day t only *reads* day $t-1$.

- Shared and immutable: price history, config. Both const.
- Mutable but partitioned: task t writes only `out[t]`.
- **Locks used: none.** If a parallel loop over independent data needs a lock, the split is wrong.
- Output bit-identical to serial — asserted with `==` on doubles, not a tolerance.

Follow-up: "Isn't that dodging the question?" The question asked me to parallelise it safely. I did — on the axis where it's worth doing, and I showed the measurement for why the named axis isn't. Doing it anyway and reporting a 1.017× ceiling would be following instructions rather than engineering.

The bigger win: the spec says "top gainers and losers," not a total order. `partial_sort` + `nth_element` = **2.4×**, no threads. That's ~140× more improvement than parallelising the loop could ever give.

Problem 3 — Snapshot Cache

Decision 1: There are two levels of shared state

What: the container is `unordered_map<string, vector<Tick>>` — so there are **two** independent races, with two different fixes.

Level 1 — the map. `buffers_[instrument]` for a new symbol can trigger a **rehash**, which rebuilds the bucket array and relinks every node. A reader walking a bucket chain follows pointers mid-rewrite.

Level 2 — the vector. `push_back` can reallocate, freeing the block a reader is iterating. This happens **even when the map never changes** — steady state, no new instruments, still racing.

Follow-up: "Doesn't per-instrument locking fix it?" It fixes level 2 and leaves level 1 wide open, because the map sits above that granularity. You'd have a lock protecting a vector you reached through an unsynchronised traversal.

Follow-up: "But references survive a rehash — so what's racing?" This is the subtle bit, and worth getting exactly right. `unordered_map` is **node-based**, so a rehash does *not* invalidate a reference to a mapped value. Your `auto& buf` stays valid.

What races is the **traversal to obtain it**: computing `hash % bucket_count` against a bucket count that's changing, indexing an array that was just freed, following `->next` pointers being relinked. There's no torn *value* — there's a torn *journey*.

That's precisely why it survives code review. You see `auto& buf = buffers_[sym]`, you know references survive rehash, it looks fine.

Decision 2: A read can trigger the rehash

Why it's counterintuitive: `latest()` and `average()` both used `operator[]`, which inserts. So **two calculation threads that never write anything can corrupt each other.**

That kills the natural instinct that "only the writer needs a lock."

Decision 3: Return by value

What: `std::optional<Tick>` instead of `const Tick&`.

Why: the `lock_guard` is destroyed at `return`. The caller then holds a pointer into a buffer the feed thread can reallocate at will. The lock bought nothing — like locking the door and posting the key through the letterbox.

The best property of this bug: it's testable **without concurrency**. Take the value, push 100,000 ticks to force reallocation, read it back. Guaranteed ASan use-after-free, every single run, in under a millisecond.

Say the general lesson: where you can turn a race into a lifetime bug, do it. A test that fails 100% of the time is

infinitely cheaper to debug than a heisenbug.

Follow-up: "Isn't copying slower?" `Tick` is 16 bytes of trivially-copyable data — it returns in two registers. There is no copy in any meaningful sense.

Decision 4: The ring buffer

Why: `erase(buf.begin())` doesn't cheaply delete — it **memmoves the entire buffer down one slot**, because `vector` must stay contiguous. Every tick, once full.

capacity	original	ring
16	15.1 ns	11.5 ns
256	55.9 ns	10.7 ns
1024	**135.6 ns**	**10.7 ns**

The shape matters more than the numbers: the original scales with how much history you keep, so *configuration becomes a latency knob*. Someone widens the window from 64 to 1024 for a better signal and quietly quadruples feed-thread latency.

And it's jitter, at the worst moment. The O(N) cost only appears once buffers *fill* — so every instrument independently gets slower a few seconds after it starts ticking, which at market open is exactly when rates peak.

Follow-up: "Explain the index arithmetic."

```
        buf[(head + size - count + i) % size]
```

`head` is one past the newest, so stepping back `count` lands on the oldest, then `+i` walks forward. **The `+ size` is load-bearing** — `head - count` can go negative, and on `size_t` that wraps to ~1.8×10^{19}, producing a valid-looking but *wrong* index. Silent corruption, not a crash.

Decision 5: The bug that survives correct locking

What: calling `average()` then `latest()` separately.

Each call is individually correct. Each is properly locked. **And the pair is wrong**, because the writer moves between them — so the two numbers describe different moments.

Number, on the fully-corrected code: 114 invariant violations in 3 million reads. Zero using a single `snapshot()` call.

That's ~1 in 27,000 — rare enough to survive every test you'd naturally write, frequent enough to fire many times a day.

The general principle: thread-safe methods do not compose into a thread-safe transaction. Correctness only extends as far as the lock does. A caller cannot build this from outside, so the API has to offer it.

This is my candidate for the reported "stale prices" — because no amount of examining the locking finds it. The locking is fine.

Decision 6: Seqlock over mutex (the Tier-2 version)

The decisive argument is not speed — it's who can block whom. Under any mutex, a reader descheduled by the kernel while holding the lock stalls the **feed thread** for a full scheduling quantum. The NIC ring backs up and you drop packets.

Why not `shared_mutex`: acquiring a *shared* lock still **writes** a shared counter, so N readers on N cores ping-pong one cache line. Often slower than a plain mutex for short critical sections.

The cost, stated honestly: readers can retry and, under an extreme write rate, give up. That's a regression in the reader's worst case, traded for the writer never blocking. The give-up count must be a **monitored metric**, not a silent `continue`.

Follow-up: "How do you verify it?" Not by stress duration — a race made 100× rarer looks identical to one that's fixed. In order: compile-time `GUARDED_BY` annotations (a proof over all paths); a model checker for the lock-free part; ThreadSanitizer — **which cannot model standalone fences**, so it can't validate the seqlock itself; stress testing as corroboration.

And run every check against the old code first. If TSan doesn't flag the original, your harness isn't reaching the racy path.

Problem 4 — P&L and Terminology

The numbers

Position	70 shares, avg ₹2,453.33
Realised	**₹1,333.33**
Unrealised @ 2480	**₹1,866.67**
Total	**₹3,200.00**
Gross traded notional	**₹565,600**
Net notional @ mark	**₹173,600**

The two one-liners

> ***Realised P&L is booked when a fill reduces the absolute size of the position*** *— a partial close, a full close, or the closing leg of a flip. Never on an open or an add.*

> ***The remaining position is marked against the externally supplied mark*** *— not the last fill price, not its own entry price. Entry price determines cost; the mark determines value.*

The note's errors — I count five, not four

1. **"70 lots" → 70 shares.** Cash equity has no lot size. At a lot size of 500 that's ₹8.68 crore instead of ₹1.74 lakh — **500× from one wrong noun**.
2. **"₹565,600 exposure"** is gross *traded* notional (turnover, monotonic). Net exposure is `70 × 2480 = ₹173,600`. **Overstates 3.3×**, widening all day.
3. **"₹3,200 realised"** — this is the sharpest one. **₹3,200 is exactly total P&L** (1,333.33 + 1,866.67). They reported total as realised.
4. **"delta 70%"** — cash equity delta is **1.0 by definition**. Position delta is 70 *shares*.
5. **"filled at VWAP so slippage was zero"** → buy VWAP ₹2,453.33 vs arrival ₹2,450 = **+13.6 bps adverse**. And slippage is undefined without naming a benchmark.

The pattern worth pointing out: every error is *directionally flattering* — bigger position, bigger book, more banked profit, better execution. That's the tell.

Follow-up: "The question says four." I make it five and I'd rather over-report. The "lots" slip may have been folded into the exposure error, but it's a distinct mistake with its own magnitude.

Decision: net delta is 1 − DF, not zero

Number: +0.005737.

Why: the options pair is a synthetic short worth `−DF` futures, not `−1`, because the premium is paid today and settles at expiry. Against the long future's +1, that leaves 0.57%.

Why it doesn't matter: it's a *mark-to-market* delta, not terminal exposure. The expiry payoff is a constant whatever the index does. It's precisely the financing on the 36-point premium — `36 × (e^rT − 1) = 0.21 points` — and it decays to zero.

Vega is exactly zero (5,514.81 each leg, cancelling), so an IV move after the trade is on cannot touch the locked P&L.

Decision: total is convention-independent, the split isn't

Convention	Realised	Unrealised	Total
Weighted average	₹1,333.33	₹1,866.67	**₹3,200**
FIFO	₹1,600.00	₹1,600.00	**₹3,200**
LIFO	₹1,100.00	₹2,100.00	**₹3,200**

India mandates FIFO for capital gains on listed equity, so a weighted-average risk book will *correctly* disagree with the tax book. **Reconcile on total, never on the split.**

Problem 5 — Synthetic Future Arbitrage

Decision 1: `e^{+rT}`, not `e^{−rT}`

```
F_syn = K + (C − P) × e^{+rT}
```

Why: you pay the premium **today**; the futures P&L settles at **expiry**. So the premium difference must be carried **forward** to be comparable with a futures price.

At these numbers, getting it backwards misprices by 0.4 points — small enough to survive review, large enough to matter against a 5-point spread.

Decision 2: Check the neighbouring strikes first

K	F_syn (mid)	vs future
48,400	48,499.57	−0.43
48,500	**48,540.23**	**+40.23**
48,600	48,500.43	+0.43

Why this matters: 48,400 and 48,600 agree to within half a point — inside their own spreads. A wrong rate or a mispriced future would move **all three together**. So this is one dislocated option pair, not a model error.

Say that you checked. Believing a 40-point dislocation without ruling out your own inputs is the mistake.

Decision 3: Executable prices only

```
mid-based "edge"      40.2308
executable edge       31.2077
difference             9.0231
```

And 9.0231 reconciles **exactly** to `(call half + put half) × e^rT + future half` = 4.0231 + 5.0000.

22% of the apparent edge is spread. If that reconciliation didn't come out exact, you'd know a side was backwards.

Decision 4: The sanity band

```
        if (edge > maxPlausibleEdge_) continue;   // too good, so bad data
```

A 200-point conversion on liquid BankNifty is a stale quote, not free money. **Trading on bad data loses; missing a real trade doesn't.** Same discipline as the ranking engine's return band.

Decision 5: The verification

2,000 arbitrage-free markets — built by picking a synthetic price and setting every put so parity holds exactly, then widening by random spreads. **Zero false alarms.**

1,000 planted mispricings — 0 missed, 0 mis-sized or mis-signed.

A detector that fires on a fair book is worse than useless, and building the book from the formula guarantees it really is fair.

The (c) answers

1. No IV dependence. Repricing the ATM pair from 5% to 60% vol moves the call 276 → 3,305 points while `C − P` stays at **0.000000**. Parity is static replication, no distribution enters.

2. Vega exactly zero, delta 1 − DF. Covered above.

3. The IV gap *is* the forward mispricing. Call 9.429% vs put 8.703% — a **72.5 bps gap** where neighbours are under 1 bp. Reprice off `F_syn` and it collapses to **exactly 0.00 bps, both at 9.058%**.

Still an opportunity because you trade in **prices, not vols** — you never need to know which leg is wrong. What parity *can't* tell you is the decomposition; for that you need the neighbouring strikes.

Cross-Cutting Themes

If they ask a general question, these are the threads that run through everything.

1. Measure, don't assume

Every claim has a number. And three of my own benchmarks lied before I caught them:

- A "packed struct" optimisation that was **2× slower** than what it replaced
- **Constant input** making the engine look 4× faster — the sort's data arrived pre-sorted
- **Hot repeated data** making the sort look 5× faster — the branch predictor memorised the pattern

All three flattered the code. **A benchmark that flatters you is the failure mode to design against.** The tell was component timings not adding up to end-to-end — and chasing that gap found both the harness bugs and a real layout bug in my own code.

2. Silent wrongness beats loud failure — as a danger

Ranked in order of how much they'd worry me:

1. **NaN propagating** — every comparison false, so `if (price > limit)` quietly stops firing
2. `+inf` **in a ranking** — sorts perfectly, declares a non-trading stock top gainer
3. **A mutex giving 30 answers** — no TSan warning, correct share counts, wrong money
4. **The compose bug** — 114 per 3 million, individually-correct locked calls

A crash gets fixed Tuesday. These don't.

3. Question the requirement before optimising it

- P2: "top gainers and losers" isn't a total order → **2.4×**, no threads
- P2: parallelising the named loop → **46–82× slower**
- P1: batching the binding → **1.1× becomes 5.3×**

4. Verification has a hierarchy

Compile-time proof > model checking > sanitizers > stress testing. **"It stopped reproducing" is not evidence.** And run every check against the broken code first — a test that's never failed on known-broken code has demonstrated nothing.

5. Know what you'd do differently

Have these ready — admitting them is a strength:

- I made the **same layout mistake** I'd just criticised, in my own carried state
- My first **packed-struct optimisation was 2× slower**
- My first **stress test measured the wrong thing** (composed calls), which is how I found the compose bug — but it was luck, not design
- The **arena allocator** I built for batch output was **within noise**; I kept it for bounded latency, not throughput, and said so